A Multimodal, Infrastructure-Free Offline Emergency Communication System for Mobile Devices Using Adaptive Hybrid Mesh Networking

Sarkar Sifatullah Haque Sajeeb[1], Md. Yousuf Hossain[1], S M Nabil Ausaf[1], Md. Nazmul Huda Masud[2], Md. Fazlay Rabby[3], and Md. Abdul Based[1]

[1] Department of Computer Science and Engineering,
Dhaka International University, Dhaka, Bangladesh
[2] Institute of Information Technology (IIT),
University of Dhaka, Dhaka, Bangladesh
[3] Faculty of Computer Science and Engineering,
Patuakhali Science & Technology University (PSTU), Bangladesh
{sifatulla.haque, itsmemehrab369, nabilausaf0}@gmail.com, nh_masud@yahoo.com,
fazlaycse@gmail.com, based@diu.ac

Abstract. Communication infrastructure often fails exactly when disasters create peak demand. We present GetConnect, an infrastructure-free emergency communication system that enables text and voice messaging on smartphones without cellular or Internet access. The system combines Bluetooth Low Energy for energy-efficient peer discovery, Wi-Fi Direct for higher-throughput transfer, and store-and-forward multi-hop routing. Implemented in Flutter, it supports Android and iOS and secures payloads with AES-256-GCM. We further formalize an adaptive routing metric that jointly optimizes hop distance, link quality, and relay battery state. Physical experiments on 25 devices show 93.2% text delivery, 90.7% voice delivery, 1.8 s median two-hop latency, and 42 mAh/hr idle draw (approximately 24-hour operation on a 2500 mAh battery). To address scale, we run controlled trace-driven emulation up to 250 nodes under identical conditions for GetConnect, AODV, and OLSR. At 250 nodes, GetConnect reaches 92.6% packet delivery ratio, outperforming AODV (84.7%) and OLSR (81.9%) with lower energy per delivered message. These results indicate practical feasibility and improved robustness for large-scale emergency communication scenarios.

Keywords: Emergency Communication · Mesh Networking · Mobile Ad-Hoc Networks · Disaster Response · Offline Communication · Peer-to-Peer Systems

1 Introduction

The natural calamities and the massive infrastructure breakdown reveal the susceptibility of the present-day communication structures. Hurricane Katrina and the 2010 Haiti earthquake showed that cellular networks and emergency communication infrastructure would fail during crisis situations [1, 2]. These incidents point to the high dependency on centralized infrastructure, such as cellular base stations, electric grids and internet backbones, which serve as single points of failure in case of an emergency.

Despite the widespread adoption of smartphones–exceeding 6.8 billion users globally [3] most mobile applications depend on continuous internet connectivity and become ineffective when infrastructure is unavailable. However, modern smartphones are equipped with significant computational

resources and multiple wireless interfaces, making them suitable platforms for decentralized communication.

Mobile Ad-Hoc Networks (MANETs) and Delay-Tolerant Networking (DTN) have been widely studied as infrastructure-free communication paradigms [4, 5]. However, its practical application is still limited because of the problems related to energy consumption, reliability of routing in the dynamic environment, heterogeneity of the devices, security issues, and barriers to usability. The recent progress made on mobile hardware and cross platform frameworks is now allowing practical emergency communication systems to be deployed on commodity smartphones.

1.1 Problem Statement and Challenges

The development of the emergency communication system that targets practical implementation via a smartphone has a number of challenges:

Infrastructure Independence: The system should run in a fully peer-to-peer without being dependent on the cellular networks or centralized services.

Multi-hop Routing: The distance of wireless is not very high hence, good routing plans are necessary to transmit the message over a wider area.

Network Dynamics: Emergencies are extremely dynamic and topology adjustments should be changing as quickly as possible.

Multimodal Communication: Both the voice and text messaging needs to be provided to meet the various communication needs.

Resource Constraints: Communication is necessary to maintain an operation that is energy efficient [in cases of] prolonged outages.

Security and Privacy: Secure modes of communication should be provided to ensure that confidential information is secured.

Practical Deployability: The solution should be in a position to work with the off-the-shelf consumer devices with no special hardware.

1.2 Research Contributions

In the paper, I introduce a project, known as *GetConnect*, which is an emergency communication infrastructure-free smartphone system. The key deliverables can be summed up as:

- A multi-mode mesh networking architecture that utilizes a blend of the Bluetooth Low Energy to execute the discovery process and Wi-Fi Direct to execute the data transmission process.
- An adaptive dynamic distance-vector routing protocol modified to dynamic and resource constrained environment.
- Multimodal Communication to Enhance Text and Voice Messaging (Store and Forward Capability).
- end-to-end secure communication based on AES-256-GCM encryption.
- To be implemented with Android and Ios devices; Cross platform implementation in the use of Flutter.
- Thorough testing of the experiments with regard to reliability, low latency and energy efficiency.

1.3 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the system architecture. Section 4 describes implementation details. Section 5 explains the routing protocol. Section 6 discusses security mechanisms. Section 7 presents experimental results. Section 8 discusses limitations and insights, and Section 9 concludes the paper.

2 Related Work

2.1 Emergency Communication Systems

Massive infrastructure breakdowns have seen a lot of attention given to emergency communication. As revealed by Townsend and Moss [1], centralized telecommunications systems are some of the sources of failure during disaster events. The number of workarounds systems available to overcome this is many.

FireChat [6] is an offline messaging platform that used peer-to-peer connectivity, but experienced poor routing performance and platform limits. Serval Project [7] has shown strong mesh networking with developed Android firmware, but due to its need to modify the devices used, it cannot be effectively used in practice. Briar [8] offers good security guarantees with Bluetooth and WiFi, but does not focus on latency or energy efficiency at the expense of privacy . Bridgefy [9] available under the name provides cross-platform offline messaging, but the security weaknesses found in recent studies cast doubts on its reliability, as does the name [10].

2.2 Mobile Ad-Hoc Network Routing

Hybrid approaches such as ZRP [11] and Fisheye State Routing [12] combine proactive and reactive mechanisms to balance performance and overhead. Our solution builds on the hybrid routing ideas but renders them more optimal both in terms of resource choices in smart phones and highly dynamic emergency situations.

2.3 Delay Tolerant Networking

Delay Tolerant Networking (DTN) [13] addresses communication scenarios with intermittent connectivity through store-and-forward mechanisms. Epidemic routing [14] ensures delivery through flooding but incurs high overhead. Spray-and-Wait [15] is more efficient as it works by minimizing the number of copies of a message being sent, whereas PRoPHET [16] also works by minimizing the number of copies of a message being forwarded by using encounter history.

Our system uses concepts of DTN particularly opportunistic forwarding and buffering but varies such concepts to very dynamic emergency scenarios where information about the past might not hold true.

2.4 Wireless Technologies

Mobile mesh networking has Wi-Fi Direct and Bluetooth Low Energy (BLE) which offer complementary features. BLE has a low power consumption, moderate range, and low bandwidth as

compared to Wi-Fi Direct [17], which has high throughput but consumes a lot of energy as compared to Wi-Fi Direct [18] and Wi-Fi Direct respectively. Hybrid solutions based on the use of BLE to discover devices and WiFi to exchange the data have shown better efficiency in usage [19, 20].

Our system is an extension of hybrid designs that also incorporates adaptive switching strategies dependent on network conditions, message characteristics and device energy state.

3 System Architecture

3.1 Design Principles

Our solution will be based on the principles of hybrid routing, but it will be smarter and efficient with regard to smartphone resources and emergency settings that can significantly change in a short period.

Infrastructure Independence: The architecture ensures that it can use a complete peer-to-peer architecture without using centralised servers, internet connection or cellular networks.

Graceful Degradation: Communication is not broken at once as density of a network is increased gradually.

Energy Efficiency: Policies are modified to facilitate a long-term running under restricted battery conditions.

Automatic Configuration: The system is automatic and does not take much interaction by a user.

Security by Design: End-to-end encryption and authentication are integrated into the architecture.

Cross-Platform Support: Stable functionality is provided in Android and iOS platforms.

3.2 Layered Architecture

The system follows a four-layer architecture.

Presentation Layer: Implemented using Flutter, this layer provides interfaces for messaging, peer monitoring, and system interaction, emphasizing simplicity and usability under stress.

Application Logic Layer: Application state is managed using the Provider pattern [21]. NetworkProvider is responsible for network and routing state and MessageProvider is responsible for message handling and status of message delivery so that we can have separation between UI and logic.

Core Services Layer: Basic functionality sealed up in four services:

- **NetworkService** manages peer discovery, routing maintenance, message forwarding, and connection lifecycle.
- **EncryptionService** provides end to end encryption (using AES-256-GCM), and the management of secure key exchange.
- **AudioService** supports not only recording, compressing, encryption and playback of voice messages.
- **StorageService** persists messages, peer data, and keys using local storage with automatic cleanup policies.

Network Communication Layer: This layer interfaces with platform-specific Bluetooth and Wi-Fi APIs, abstracting underlying differences to provide a unified communication interface.

3.3 Hybrid Mesh Network Architecture

The suggested architecture would use the hybrid approach of wireless that leverages Bluetooth Low Energy (BLE) to discover low-power peers and Wi-Fi Direct (TCP over port 8080) to share data over high bandwidth. This isolation allows the use of continuous neighbor detection at a minimal energy cost and leave Wi-Fi to data-intensive activities such as voice communication.

Peer Discovery Devices periodically broadcast BLE advertisements containing a compact metadata payload: device UUID (v4), user-defined name, supported capabilities (text, voice, relay), battery level, protocol version, and timestamp.

After identifying a new advert, a TCP connection is established to exchange longer peer information such as IP address, routing table summary, topology knowledge, encryption material, and the quality of links. The two phase discovery mechanism is a tremendous power saving approach since it restricts the costly Wi-Fi transmission only to new peers.

Connection Management Each node maintains three logical peer sets:

Discovered Peers: Nearby devices detected via BLE but without active TCP sessions. Signal strength is estimated from BLE RSSI values.

Connected Peers: One-hop neighbors with active bidirectional TCP connections used for data transfer and routing updates.

Reachable Peers: Multi-hop devices accessible through routing table entries containing next-hop, hop count, and link-quality metrics.

Connections are established on demand and automatically terminated after 30 seconds of inactivity to conserve energy. Re-establishment latency remains below 500 ms in typical conditions.

Message Flow End-to-end communication has the paradigm of store-and-forward. Messages are encrypted with the symmetric key of the recipient after composition and passed on based on the results of routing tables lookups. When a first-hand neighbor is present, the message will be sent on-the-fly via TCP with an acknowledgement option. Otherwise the packet is forwarded via the chosen next-hop node where the intermediate devices increase the hop count and pass onwards to the destination. On reaching the receiver, the receiver decrypts the payload and sends a confirmation back to the sender.

3.4 Protocol Design

The system defines six lightweight control and data message types:

- **DISCOVERY_REQUEST / DISCOVERY_RESPONSE** – peer identification
- **ROUTING_UPDATE** – routing table dissemination
- **MESSAGE** – encrypted user payload
- **MESSAGE_ACK** – delivery confirmation
- **HEARTBEAT** – liveness monitoring (5 s interval)
- **RELAY_REQUEST** – multi-hop forwarding

All protocol messages are serialized in JSON format and transported over TCP to ensure reliability and ordered delivery.

4 Implementation

4.1 Cross-Platform Architecture

GetConnect is implemented using Flutter 3.8.1 [22,23], enabling a unified Dart codebase for Android and iOS deployment. Flutter compiles to native ARM binaries, providing near-native performance while significantly reducing development overhead compared to dual-native implementations.

It has a framework that has direct access to platform specific networking via platform channels, through which the framework interacts with low-level, BLE discovery and Wi-Fi Direct APIs. Such a hybrid solution also has the advantage of permitting performance-sensitive processes (networking and cryptography) but providing cross-platform compatibility.

Key external packages include `connectivity_plus` and `network_info_plus` (network state monitoring), `just_audio` (voice playback), `encrypt` and `crypto` (AES-256-GCM operations), `shared_preferences` (persistent storage), and `permission_handler` (runtime permissions).

4.2 Core Services

The system is based on a modular service-oriented design based on four main singleton services.

NetworkService: Introduces peer discovery, maintenance of routing, routing of messages and control of TCP sessions. The service obtains runtime permissions during the process of initializing, creates a persistent UUID (v4) and launches a TCP service on port 8080. Discovery is also done in periodic cycles between BLE scanning and subnet probing. Messages are forwarded through adaptive multi-hop which enforces hop limits and TTL. Weaknesses in transmission cause route re-computation.

EncryptionService: Provides end-to-end authenticated encryption using AES-256-GCM with a random 16-byte IV per message. For prototype purposes, shared secrets are derived from exchanged ephemeral keys; future production versions will integrate full ECDH-based key exchange to achieve forward secrecy.

AudioService: Handles voice capture, compression, encryption, and playback. Audio is recorded at 16 kHz mono and encoded using AAC at adaptive bitrates (8–32 kbps). Payloads are segmented into 4 KB encrypted frames to support reliable multi-hop streaming. Bitrate adaptation considers hop count, link quality, and battery level.

StorageService: Manages persistent state, storing message metadata using `SharedPreferences` and voice payloads on the filesystem. Messages include delivery state, retry counters, and 24-hour TTL. Automatic cleanup imposes a storage cap of 7 days and a storage size of 100MB to ensure uncontrolled growth.

4.3 Platform Constraints

Platform-level restrictions can have a great impact on deployment.

Android allows background networking but must have battery optimization exemption (Doze mode) to continue to network on a mesh. There is Wi-Fi Direct APIs, which needs location permissions.

iOS puts more severe background execution limits, limiting BLE scanning and TCP connectivity during suspension of the application. There are no Wi-Fi Direct APIs and, as such, connectivity depends on BLE and special entitlement in the Network Extension framework. Background networking also restricts a user to around three minutes background networking.

Algorithm 1 Peer Discovery Process

```
Initialize discovered_peers ← ∅
Determine local subnet from IP address
for each ip ∈ subnet do
    Attempt TCP connection on port 8080
    if connection successful then
        Send DISCOVERY_REQUEST
        Await DISCOVERY_RESPONSE with timeout
        if response received then
            Add peer to discovered_peers
            Estimate signal strength from response time
            Emit peer_discovered event
        end if
    end if
end for
Sort discovered_peers by signal strength
return discovered_peers
```

Algorithm 2 Message Relay Algorithm

```
Receive message from sender
Extract recipientId, hopCount
if recipientId = localDeviceId then
    Decrypt and deliver message locally
    Send MESSAGE_ACK to sender
    return
end if
if hopCount > MAX_HOPS then
    Drop message                                  ▷ Prevents infinite loops
    return
end if
Look up route to recipientId in routing table
if route exists then
    nextHopId ← route.nextHop
    hopCount ← hopCount + 1
    Forward message to nextHopId
else
    Buffer message                                ▷ Store-and-forward mechanism
    Wait for routing update
end if
```

5 Adaptive Routing Protocol

5.1 Protocol Design

The proposed routing mechanism extends distance-vector principles with energy- and link-aware adaptations tailored for dynamic emergency environments.

Routing Table Structure. Each entry maintains: destination ID, next-hop, hop distance (1–10), composite metric, link quality, next-hop battery level, and timestamp for staleness detection.

Composite Routing Metric. Route selection is based on a weighted objective function:

$$metric = 50 \cdot d + 100 \cdot (1 - q) + 100 \cdot (1 - b) \tag{1}$$

where d denotes hop distance, $q \in [0, 1]$ represents link quality, and $b \in [0, 1]$ is the normalized battery fraction of the next-hop node.

The weights in Eq. (1) are selected through grid-search tuning on development traces to maximize packet delivery ratio (PDR) under an energy constraint. The objective is to minimize route failures caused by unstable links and battery depletion at relay nodes. Therefore, $(1 - q)$ and $(1 - b)$ are weighted more strongly than distance: short routes are not preferred if they are fragile or energy-critical. This formulation provides a practical trade-off between reliability, latency, and network lifetime, and is intentionally lightweight for real-time execution on smartphones.

Route Discovery and Update. Direct neighbors are initialized with $d = 1$. Upon receiving routing updates from peers, nodes increment the advertised hop count, recompute the metric using Eq. (1), and retain routes with lower metric values. Updated entries trigger dissemination to adjacent peers, enabling distributed topology convergence.

Algorithm 3 Adaptive Routing Table Construction

```
Initialize routingTable ← ∅
for each peer ∈ connectedPeers do
    distance ← 1
    metric ← calculateMetric(distance, peer.linkQuality, peer.battery)
    Add route to routingTable
end for
Request routing tables from all connected peers
for each routingUpdate received from peer do
    for each entry ∈ routingUpdate.routes do
        newDistance ← entry.distance + 1
        newMetric ← calculateMetric(newDistance, peer.linkQuality, peer.battery)
        if target not in routingTable or newMetric < currentMetric then
            Update routingTable entry
            nextHop ← peer
        end if
    end for
end for
Broadcast updated routing table to neighbors
```

5.2 Routing Maintenance

Routing state is maintained through lightweight periodic exchanges.

Heartbeat Mechanism. Peers connected to each other send heartbeat messages on a 5 s interval with peer ID, timestamp, and battery level. A missed interval triggers a warning state, three consecutive misses (15 s) invalidates the route. Heartbeats can also be used to give liveness and dynamic battery information.

Routing Updates. Every 10 s, nodes broadcast compressed routing tables to direct neighbors (2–8 KB for 25–50 peers). Upon reception, entries are merged and metrics recomputed using Eq. (1). Superior routes replace existing entries, enabling proactive convergence.

Link Quality Estimation. Link quality is computed as

$$q = 0.3\, RTT_{norm} + 0.5\,(1 - lossRate) + 0.2\, stability \tag{2}$$

where $RTT_{norm} \in [0, 1]$ denotes normalized round-trip delay, $lossRate$ represents packet loss over 60 s, and $stability$ captures connection duration. The weighting prioritizes delivery reliability over latency.

Route Aging. Routes without updates for 30 s are marked stale; after 60 s they are removed, preventing outdated paths from persisting under topology changes.

5.3 Loop Prevention

To reduce the problems of classical distance vector loops, 4 mechanisms are used:

- **Split Horizon** does not allow to advertise routes towards the source neighbor.
- **Maximum Hop Limit** deletes the routes that have more than 10 hops.
- **Sequence Numbers** compress obsolete routing information.
- **Path Vectors** This algorithm blocks advertisements that contain the ID of the local node.

The combination approach provides bounded approach convergence and loop resilience in dynamic environments.

5.4 Store-and-Forward

When no route exists, messages are buffered locally instead of discarded. Each node maintains a priority queue (capacity 100) with emergency messages prioritized. Retransmissions follow exponential backoff (10 s, 30 s, 90 s, 270 s) until delivery or expiration. Messages carry a 24-hour TTL to prevent indefinite propagation. This opportunistic forwarding significantly improves reliability in sparse and mobile deployments (Section 7).

6 Security and Privacy

6.1 Threat Model

The system takes into account passive eavesdropping, message tampering, replay attack, impersonation, denial-of-service and malicious relaying. We also make assumptions of integrity in devices used in emergency and proper implementation of cryptographic primitives.

6.2 Defense Mechanisms

End-to-End Encryption. Every user payload is encrypted with AES-256-GCM, which grants confidentiality and integrity with authentication. Encryption is done at the source and relay nodes are not able to see plaintext.

Replay Protection. Messages consist of sender UUID, timestamp and sequence-number. Separate packets are discarded by the recipients by a sliding acceptance window.

Identity Verification. Peer verification is optional, but based on the use of QR-based fingerprint exchange, which is based on SHA-256 identity hash.

Rate Limiting. Peer-to-peer transmission is limited to a maximum of 100 messages/minutes in order to prevent attacks of flooding.

6.3 Privacy Considerations

Packages are encrypted, but routing metadata (sender, recipient, size, timestamp) can still be seen by the intermediate relays. This trade-off is biased towards performance and not complete anonymity. Messages are removed successfully at the time of delivery or TTL has expired (24 h), and there is no centralized storage and synchronization into the cloud. The individuals can use pseudonyms without giving personal identifiable information.

7 Evaluation

7.1 Experimental Setup

We evaluated the system on 25 smartphones: 15 Android (Galaxy S21/S22, Pixel 6/7, OnePlus 9) and 10 iOS (iPhone 12–14), deployed in a 3-floor 5000 m^2 university building with concrete partitions and mixed room sizes.

Four scenarios were tested:

- **Dense:** 25 devices within 1000 m^2 (avg. 3.2 neighbors).
- **Sparse:** 25 devices across 5000 m^2 (avg. 1.5 neighbors).
- **Mobile:** Random waypoint movement (1.4 m/s).
- **Partitioned:** Two clusters separated by 200 m requiring 4-hop relays.

Each scenario transmitted 1,250 messages (text ∼100B, voice ∼85KB), totaling 5,000 messages.

7.2 Message Delivery Performance

The averaged text delivery was `93.2%` and voice `90.7%`. There is a tradeoff between reliability and hop count and payload size, but even worst-case partitioned deployment had success of approximately 90%. Thick networks are advantageous in having redundant routes whereas thin cases promote the usefulness of store-and-forward buffering. The mobile environments enhanced communication via chance occurrences.

The convergence ranged between 14 s (dense) and 48 s (partitioned) and was therefore still sensibly acceptable to initialize an emergency.

Table 1: Message Delivery Performance

Scenario	Text	Voice	Avg Hops	Conv.
Dense	97.2%	95.8%	1.9	14s
Sparse	92.7%	89.4%	3.2	31s
Mobile	93.8%	91.1%	2.4	22s
Partitioned	89.1%	86.3%	4.6	48s
Average	**93.2%**	**90.7%**	**3.0**	**29s**

Table 2: Power Consumption (2-hour average)

Activity	Android (mAh/hr)	iOS (mAh/hr)
Idle (connected)	42	36
Active discovery	175	158
Sending	118	106
Relaying	92	81

7.3 End-to-End Latency

Median latency scales approximately linearly with hop count:

- 1-hop: 380 ms
- 2-hop: 1.8 s
- 3-hop: 3.1 s
- 4-hop: 5.4 s

Per-hop overhead averages 1.43 s including routing lookup, TCP setup, encryption (≈3 ms), and transmission. The 90th percentiles latency is still less than 8.2 s for 4 hop paths. Text traffic has little congestion sensitivity, Voice latency increase 15∼25% under load because of payload size.

7.4 Power Consumption

Idle connectivity enables ≈24 hours of operation on a 2500 mAh battery. Discovery is energy-intensive but short-lived. Message relaying incurs modest overhead, supporting sustainable mesh participation. iOS demonstrated ∼12% improved efficiency due to background task optimization.

7.5 Scalability

To evaluate scalability beyond the 25-device field deployment, we conducted controlled trace-driven emulation at 50, 100, 150, 200, and 250 nodes. The emulation uses the same traffic mix, message format, and calibrated link-loss model derived from field logs, following MANET evaluation best practices [24, 25]. Routing table size grows linearly (11 KB at 50 nodes to 58 KB at 250 nodes), while convergence time scales from 27 s to 74 s. Queue pressure and tail latency increase above 200 nodes, but routing overhead remains below 9% of available Wi-Fi Direct throughput.

Table 3: Controlled Protocol Benchmark at 250 Nodes (Identical Conditions)

Protocol	PDR (%)	P95 Latency (s)	Energy/msg (J)
AODV [26]	84.7	9.8	2.46
OLSR [27]	81.9	8.9	2.71
GetConnect (ours)	92.6	7.1	1.88

The benchmark uses identical mobility traces, transmit power, channel model, and offered load for all three protocols. GetConnect shows higher reliability and lower energy per delivered message due to its energy- and link-aware metric and store-and-forward operation under intermittent connectivity.

7.6 Voice Quality

The voice quality assessed using PESQ was between 4.0 (1-hop) and 2.7 (4-hop). The use of adaptive bitrate reduction (32-8 kbps) maintains the intelligibility in the case of multi-hop. Even poor audio was understandable in case of emergency communication.

7.7 Comparative Analysis

FireChat [6] achieved similar delivery rates but relied on platform-specific frameworks. **Briar** [8] offers stronger anonymity via Tor at higher latency. **Bridgefy** exhibited security vulnerabilities [9].

Key Contribution: The proposed system combines cross-platform deployment, end-to-end authenticated encryption, and adaptive routing with controlled benchmark evidence against AODV and OLSR under identical conditions.

8 Discussion

8.1 Design Insights

Hybrid Wireless Architecture. The effective trade-off between energy efficiency and bandwidth was developed with BLE being used to implement low-power discovery plus Wi-Fi Direct being used to implement high-throughput data transfer. Wi-Fi only solutions depleted too much scanning power and BLE-only solutions had insufficient throughput to transmit voice. The hybrid model takes advantage of the low-energy continuous discovery of BLE and the multi-megabit of the Wi-Fi data.

Store-and-Forward Impact. Direct-delivery-only prototypes achieved 78% success in sparse deployments. Introducing store-and-forward buffering increased delivery to 93%, confirming that opportunistic forwarding is essential in dynamic and intermittently connected emergency topologies.

Routing Algorithm Choice. GetConnect outperformed AODV and OLSR in controlled 250-node benchmarking (PDR 92.6% vs 84.7% and 81.9%, respectively). AODV incurred route-repair delays under frequent topology changes, while OLSR introduced higher control overhead and energy draw. In dynamic emergency topologies, combining lightweight distance-vector logic with adaptive link/energy weighting improved robustness.

Cross-Platform Considerations. Flutter enabled rapid cross-platform development; however, platform-specific networking (particularly iOS background restrictions) required significant customization. The nature of iOS regarding sustained networking in the background makes persistent networking asymmetric to Android devices.

8.2 Limitations and Future Work

Physical Range. Wireless range (10–100 m BLE; 100–200 m Wi-Fi Direct) constrains coverage in large-scale disasters. It can be expanded in the future, with aerial relay nodes (e.g., drones) used to cross between clusters.

Platform Constraints. Background network limits in iOS decrease in background mesh stability. The continuity of backbones of hybrid deployments might depend more on Android devices.

Network Partitions. Total physical isolation of clusters is done so that they cannot communicate until the mobility closes distances. Further research is required on strategic relay deployment mechanisms or automated partition detection mechanisms.

Security Extensions. Devices that are compromised are not covered in the current assumptions. The improvements to be implemented in future are reputation systems, better peer authentication, Byzantine-resilient routing and forward-secret key exchange (e.g., ECDH).

Scalability Boundaries. The largest physical deployment in this study is 25 devices; larger-scale evidence up to 250 nodes is emulation-based, calibrated from real traces. This supports scalability claims within the tested envelope, but full live validation at larger scales remains future work. Hierarchical clustering and inter-zone relays are expected to be necessary beyond 500 nodes.

Further developments are adaptive routing optimization by ML, where infrastructure gateways are available and multimedia and interoperability standards across mesh communication platforms.

9 Conclusion

GetConnect is a free emergency communication infrastructure that is based on a Smartphonebased product. The proposed system operates on an adaptive hybrid mesh networking architecture, enabling both text and voice communication. Most of its contributions have been in the form of a hybrid BLE/Wi-Fi architecture, distance-adaptive link quality aware vector routing, AES-256-GCM based end-to-end encryption and cross-platform Flutter implementation. A physical test with 25 devices reported 93.2% text delivery, 90.7% voice delivery, 1.8 s median 2-hop latency, and 42 mAh/hr idle power consumption, enabling approximately 24 hours of operation. In controlled emulation up to 250 nodes, GetConnect achieved 92.6% PDR and outperformed AODV and OLSR under identical conditions, indicating that the routing design remains effective beyond prototype scale. Despite the existence of such limitations as range and iOS limitations, partition handling etc. this work demonstrates that the infrastructure-free emergency communication is possible using the technology at hand. The peer-to-peer communication systems are important as disasters escalate more and more and the infrastructure is more prone to destruction. GetConnect offers a deployable solution, that activates during system failure in traditional systems and allows coordination and assistance to save lives.

The system is open-source so that people can research it, contribute to it, and use it in areas that have infrastructure constraints. The future work will be more secure, will be able to interoperate with the emergency services, and will examine other modalities.

References

1. Anthony M Townsend and Mitchell L Moss. *Disrupted Networks: From Physics to Climate Change*. MIT Press, Cambridge, MA, 2013.
2. Louise K Comfort and Naim Kapucu. Crisis information management: Communication and technologies. *Journal of Information Technology & Politics*, 9(1):1–5, 2012.
3. Statista. Number of smartphone users worldwide from 2016 to 2023. https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/, 2023. Accessed: 2024-01-15.
4. C Siva Ram Murthy and B S Manoj. *Ad Hoc Wireless Networks: Architectures and Protocols*. Prentice Hall PTR, Upper Saddle River, NJ, 2004.
5. Mehran Abolhasan, Tadeusz Wysocki, and Eryk Dutkiewicz. A review of routing protocols for mobile ad hoc networks. *Ad Hoc Networks*, 2(1):1–22, 2004.
6. Open Garden Inc. Firechat: Off-the-grid messaging. https://www.opengarden.com/firechat, 2014. Accessed: 2024-01-15.
7. Paul Gardner-Stephen and Romana Palaniswamy. The serval project: Practical wireless ad-hoc mobile telecommunications. In *Linux Conference Australia*, 2011.
8. Briar Project. Briar project: Secure messaging for activists, journalists and anyone who needs a safe, easy and robust way to communicate. https://briarproject.org, 2018. Accessed: 2024-01-15.
9. Bridgefy Inc. Bridgefy: Off-grid messaging. https://www.bridgefy.me, 2016. Accessed: 2024-01-15.
10. Martin R Albrecht, Lenka Mareková, Kenneth G Scott, and Navendu Tyagi. Security analysis of the bridgefy messaging protocol. In *2020 IEEE European Symposium on Security and Privacy Workshops*, pages 573–582. IEEE, 2020.
11. Zygmunt J Haas, Marc R Pearlman, and Prince Samar. The zone routing protocol (zrp) for ad hoc networks. *Internet Draft, Internet Engineering Task Force*, 1998.
12. Guangyu Pei, Mario Gerla, and Xiaoyan Hong. Fisheye state routing: A routing scheme for ad hoc wireless networks. In *IEEE International Conference on Communications*, volume 1, pages 70–74. IEEE, 2000.
13. Kevin Fall. A delay-tolerant network architecture for challenged internets. *Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, pages 27–34, 2003.
14. Amin Vahdat and David Becker. Epidemic routing for partially connected ad hoc networks. *Technical Report CS-200006, Duke University*, 2000.
15. Thrasyvoulos Spyropoulos, Konstantinos Psounis, and Cauligi S Raghavendra. Spray and wait: An efficient routing scheme for intermittently connected mobile networks. In *Proceedings of the 2005 ACM SIGCOMM Workshop on Delay-Tolerant Networking*, pages 252–259. ACM, 2005.
16. Anders Lindgren, Avri Doria, and Olov Schelén. Probabilistic routing in intermittently connected networks. *ACM SIGMOBILE Mobile Computing and Communications Review*, 7(3):19–20, 2003.
17. Carles Gomez, Joaquim Oller, and Josep Paradells. Overview and evaluation of bluetooth low energy: An emerging low-power wireless technology. *Sensors*, 12(9):11734–11753, 2012.
18. Daniel Camps-Mur, Andres Garcia-Saavedra, and Pablo Serrano. Wifi-direct multi-group data communication and service discovery: A procedural approach. In *2013 IEEE International Conference on Communications Workshops*, pages 422–426. IEEE, 2013.
19. Yong Li, Guangjie Su, Di Wu, Depeng Jin, Li Su, and Lieguang Zeng. A hybrid bluetooth-wifi approach for opportunistic networking. In *2016 IEEE 13th International Conference on Mobile Ad Hoc and Sensor Systems*, pages 99–107. IEEE, 2016.
20. Marco Conti and Silvia Giordano. Looking ahead in pervasive computing: Challenges and opportunities in the era of cyber–physical convergence. *Pervasive and Mobile Computing*, 8(1):2–21, 2014.
21. Flutter Community. Provider: A wrapper around inheritedwidget to make them easier to use and more reusable. https://pub.dev/packages/provider, 2019. Accessed: 2024-01-15.
22. Google LLC. Flutter: Beautiful native apps in record time. https://flutter.dev, 2018. Accessed: 2024-01-15.

23. Muhammad Latif, Youness Lakhrissi, El Habib Nfaoui, and Najia Es-Sbai. Comparative analysis of cross-platform mobile application development frameworks. In *2019 International Conference on Wireless Technologies, Embedded and Intelligent Systems*, pages 1–4. IEEE, 2019.
24. Stuart Kurkowski, Tracy Camp, and Michael Colagrosso. Manet simulation studies: The incredibles. *ACM SIGMOBILE Mobile Computing and Communications Review*, 9(4):50–61, 2005.
25. David Kotz, Calvin Newport, Robert S Gray, Jason Liu, Yougu Yuan, and Chip Elliott. Experimental evaluation of wireless simulation assumptions. In *Proceedings of the 7th ACM International Symposium on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, pages 78–82. ACM, 2003.
26. Charles E Perkins and Elizabeth M Royer. Ad-hoc on-demand distance vector routing. *Proceedings of the 2nd IEEE Workshop on Mobile Computing Systems and Applications*, pages 90–100, 1999.
27. Thomas Clausen and Philippe Jacquet. Optimized link state routing protocol (olsr). *RFC 3626*, 2003.